Guía del código

Modelos de inventario: EOQ, (Q,R) y (S,T), sin y con descuentos

Explicación parte por parte del cuaderno `modelos_inventario.ipynb`

Octubre de 2026

Índice

1. Antes de empezar: cómo está armado el cuaderno **3**

2. Librerías **4**

3. Piezas comunes (sección 0) **4**

3.1. Costo de mantener . . . 4

3.2. Demanda en un horizonte . . . 5

3.3. Función de pérdida y faltantes esperados . . . 5

3.4. Tramos de precio . . . 5

4. EOQ **6**

4.1. Sin descuentos . . . 6

4.2. Con descuentos . . . 7

5. Modelo (Q,R) iterativo **9**

5.1. La función de costo . . . 9

5.2. El punto de reorden para un Q dado . . . 9

5.3. La solución completa cuando Q ya se conoce . . . 10

5.4. La iteración . . . 10

5.5. Con descuentos . . . 12

6. Modelo (S,T) iterativo **13**

6.1. Costo para un T fijo . . . 13

6.2. La búsqueda del mejor T . . . 14

6.3. Con descuentos . . . 15

7. El caso de la empresa **16**

7.1. Los datos . . . 16

7.2. Los 6 modelos con los datos reales 17

8. Gráficas del inventario en el tiempo (serruchos) 17

9. Superficies 3D de costo 19

9.1. La rejilla 19

9.2. El dibujo 20

9.3. La superficie del (S,T) 21

9.4. ¿Hasta dónde se grafica cada pedazo? (superficies por tramo) 21

10. Curvas de costo vs Q (o vs T) 24

11. Comparación y política actual 25

12. El reporte HTML 26

13. Mapa: quién llama a quién 27

14. Preguntas que te puede hacer tu asesor 28

1. Antes de empezar: cómo está armado el cuaderno

Piensa en el cuaderno como una fábrica con tres pisos. Cada piso usa lo que construyó el de abajo:

1. **Piezas comunes (sección 0).** Funciones pequeñas que todos los modelos necesitan: el costo de mantener h, la demanda en un horizonte, la función de pérdida $L(z)$ y los tramos de precio.
2. **Los modelos (secciones 1, 2 y 3).** Una función por modelo: EOQ, (Q,R) y (S,T), cada uno sin y con descuentos. Cada una recibe los datos y devuelve la política óptima con sus costos.
3. **El caso de la empresa (sección 4).** Se meten los datos reales, se resuelven los 6 modelos y se hacen las gráficas: serruchos, superficies 3D, curvas de costo, comparación y reporte HTML.

Idea clave

Ninguna función "inventa" nada: cada una es una fórmula del documento de formulación escrita en Python. Si entiendes la fórmula, entiendes la función. En esta guía, junto a cada pedazo de código, va la fórmula que implementa.

Lo mínimo de Python que necesitas

Cosa	Qué significa
`def f(a, b):`	Define una función: una "receta" que recibe datos (`a`, `b`) y devuelve un resultado con `return`.
`for j in range(n):`	Repite lo que está adentro para $j = 0, 1, \ldots, n-1$. Python cuenta desde 0.
`if / elif / else`	Decide qué hacer según una condición (por ejemplo, los casos A, B y C).
`[a, b, c]`	Una **lista**: valores en orden. `lista.append(x)` agrega `x` al final.
`{"Q": 4254, "R": 1917}`	Un **diccionario**: valores con nombre. `sol["Q"]` saca el valor llamado `"Q"`.
`None`	"Sin valor". Se usa cuando un tramo se descarta.
`f(**datos)`	"Desempaca" un diccionario: es lo mismo que escribir `f(demanda=5326, desviacion=1297, ...)`.
`f"Q = {q:,.0f}"`	Texto con números adentro (`,.0f` = con separador de miles y sin decimales).

Notación: símbolo del curso y nombre en el código

Símbolo	Significado	Nombre en el código
λ	demanda media por mes	`demanda`
σ	desviación de la demanda por mes	`desviacion`
K	costo fijo por pedido	`costo_pedido`
c	costo unitario	`costo_unitario`, `precio`
i	tasa de manejo mensual	`tasa_manejo`
$h = ic$	costo de mantener una unidad un mes	`h`
p	costo por unidad faltante	`costo_faltante`
τ	lead time (en meses)	`lead_time`
μ_τ, σ_τ	media y desviación de la demanda en el lead time	`media_lead`, `desviacion_lead`
$n(R)$	faltantes esperados por ciclo	`faltantes`

Ojo

Todo debe estar en la **misma unidad de tiempo**. Con los datos de la empresa todo está en **meses**: λ en unidades/mes, i mensual ($30\,\%/12 = 2.5\,\%$), $\tau = 5/30$ de mes.

2. Librerías

```
import math
from scipy.stats import norm   # distribución normal estándar
```

`math` trae la raíz cuadrada (`math.sqrt`), ∞ (`math.inf`) y el redondeo hacia arriba (`math.ceil`). De `scipy` solo se usa la normal estándar, con tres funciones que ya conoces de Excel:

Python	Matemática	Excel	Para qué
`norm.pdf(z)`	$\phi(z)$ (densidad)	`NORM.S.DIST(z, FALSE)`	dentro de $L(z)$
`norm.cdf(z)`	$\Phi(z) = P(Z \le z)$	`NORM.S.DIST(z, TRUE)`	dentro de $L(z)$, y α
`norm.ppf(F)`	$\Phi^{-1}(F)$ (inversa)	`NORM.S.INV(F)`	sacar z a partir de $F(R)$

Más adelante se cargan `numpy` (como `np`, para rejillas de números) y `matplotlib.pyplot` (como `plt`, para las gráficas).

3. Piezas comunes (sección 0)

3.1 Costo de mantener

```
def h_costo_mantener_una_unidad(tasa_manejo, costo_unitario):
    """h = i * c : costo de mantener una unidad por unidad de tiempo."""
    return tasa_manejo * costo_unitario
```

Es solo $h = i\,c$. Va en su propia función porque, con descuentos, cada tramo tiene su propio precio c_j y por eso su propio $h_j = i\,c_j$.

Con los números de la empresa

$h = 0.025 \times 3{,}900 = \$97.5$ por unidad al mes.

3.2 Demanda en un horizonte

```python
def demanda_horizonte_devuelve_media_y_desviacion(demanda_media, desviacion, horizonte):
    """Devuelve (mu_t, sigma_t) de la demanda en un horizonte de duración t."""
    media_horizonte = demanda_media * horizonte
    desviacion_horizonte = desviacion * math.sqrt(horizonte)
    return media_horizonte, desviacion_horizonte
```

Implementa $\mu_t = \lambda t$ y $\sigma_t = \sigma\sqrt{t}$. La media crece proporcional al tiempo; la desviación crece con la raíz porque las varianzas de periodos independientes se suman. En (Q,R) el horizonte es $t = \tau$; en (S,T) es $t = T + \tau$.

Fíjate que devuelve **dos** valores, y por eso se llama así:

```python
media_lead, desviacion_lead = demanda_horizonte_devuelve_media_y_desviacion(...)
```

Con los números de la empresa

$\mu_\tau = 5{,}325.52 \times \frac{1}{6} = 887.6$ unidades y $\sigma_\tau = 1{,}297.13 \times \sqrt{1/6} = 529.5$ unidades.

3.3 Función de pérdida y faltantes esperados

```python
def perdida_normal_L_de_z(z):
    """Función de pérdida normal estándar L(z)."""
    return norm.pdf(z) - z * (1 - norm.cdf(z))


def faltantes_esperados_n_por_ciclo(nivel, media_horizonte, desviacion_horizonte):
    """n(x) = sigma_t * L(z): faltantes esperados por ciclo si se protege hasta 'nivel'."""
    z = (nivel - media_horizonte) / desviacion_horizonte
    return desviacion_horizonte * perdida_normal_L_de_z(z)
```

- `perdida_normal_L_de_z` es $L(z) = \phi(z) - z\,(1 - \Phi(z))$.
- `faltantes_esperados_n_por_ciclo` primero estandariza, $z = (x - \mu_t)/\sigma_t$, y luego devuelve $n(x) = \sigma_t L(z)$: cuántas unidades se espera que falten en un ciclo si el nivel de protección es x. El nivel x es R en el (Q,R) y S en el (S,T).

3.4 Tramos de precio

Los tramos se escriben como una lista de pares *(desde, precio)*. Para la empresa:

```python
tramos_empresa = [(0, 3900), (4000, 3822), (9000, 3705), (15000, 3588)]
```

Es decir: menos de 4,000 unidades a \$3,900; de 4,000 a 8,999 a \$3,822 (2 %); de 9,000 a 14,999 a \$3,705 (5 %); desde 15,000 a \$3,588 (8 %).

```python
def tramo_y_precio_devuelve_j_y_precio(cantidad, tramos):
    """Devuelve (j, precio) del tramo en el que cae 'cantidad' (j empieza en 0)."""
    tramo_encontrado = 0
    for j in range(len(tramos)):
        desde = tramos[j][0]
        if cantidad >= desde:
            tramo_encontrado = j      # sigue avanzando mientras la cantidad alcance el
    tramo
    precio = tramos[tramo_encontrado][1]
    return tramo_encontrado, precio


def limites_tramo_devuelve_desde_y_hasta(j, tramos):
    """Devuelve (Q_{j-1}, Q_j): desde dónde y hasta dónde va el tramo j."""
    limite_inferior = tramos[j][0]
    if j + 1 < len(tramos):
        limite_superior = tramos[j + 1][0]
    else:
        limite_superior = math.inf    # el último tramo no tiene límite superior
    return limite_inferior, limite_superior
```

- `tramo_y_precio_devuelve_j_y_precio` recorre los tramos de menor a mayor. Mientras la cantidad alcance el "desde" del tramo, se queda con ese tramo. Al final queda el último tramo alcanzado. Por ejemplo, para 9,500 unidades pasa por los tramos 0, 1 y 2, y se queda en el 2 ($3,705).
- `limites_tramo_devuelve_desde_y_hasta` da el intervalo $[Q_{j-1}, Q_j)$ del tramo j: el límite inferior es su propio "desde" y el superior es el "desde" del siguiente. El último tramo no tiene siguiente, así que su límite es ∞.
- `tramos[j][0]` es el primer valor del par (desde) y `tramos[j][1]` el segundo (precio).

Ojo

En el código los tramos se numeran desde 0 (así cuenta Python); en el documento y en las gráficas, desde 1. El tramo `j = 3` del código es el "tramo 4" de la tesis.

4. EOQ

4.1 Sin descuentos

```python
def costos_eoq_devuelve_ordenar_adquirir_mantener_total(cantidad, demanda, costo_pedido,
    costo_unitario, h):
    """G(Q) del EOQ para una cantidad de pedido cualquiera, separado por componentes (
    diccionario)."""
    costo_ordenar = costo_pedido * demanda / cantidad
    costo_adquirir = demanda * costo_unitario
    costo_inventario = h * cantidad / 2
    return {"ordenar": costo_ordenar, "adquirir": costo_adquirir, "mantener":
    costo_inventario,
            "faltantes": 0,   # el EOQ no permite faltantes
            "total": costo_ordenar + costo_adquirir + costo_inventario + 0}
```

```python
def resolver_eoq_sin_desc_devuelve_Q_T_R_G(demanda, costo_pedido, costo_unitario,
    tasa_manejo, lead_time):
    """Resuelve el EOQ sin descuentos. Devuelve (Q*, T*, R, G*)."""
    h = h_costo_mantener_una_unidad(tasa_manejo, costo_unitario)

    q_optimo = math.sqrt(2 * costo_pedido * demanda / h)
    t_optimo = q_optimo / demanda
    punto_reorden = demanda * lead_time
    costo_optimo = costos_eoq_devuelve_ordenar_adquirir_mantener_total(q_optimo, demanda,
    costo_pedido, costo_unitario, h)["total"]

    return q_optimo, t_optimo, punto_reorden, costo_optimo
```

`costos_eoq_devuelve_ordenar_adquirir_mantener_total` es la función objetivo, término por término (y devuelve cada término por separado, en un diccionario):

$$G(Q) = \underbrace{\frac{K\lambda}{Q}}_{\text{costo_ordenar}} + \underbrace{\lambda c}_{\text{costo_adquirir}} + \underbrace{\frac{hQ}{2}}_{\text{costo_inventario}}$$

`resolver_eoq_sin_desc_devuelve_Q_T_R_G` aplica las fórmulas cerradas: $Q^* = \sqrt{2K\lambda/h}$, $T^* = Q^*/\lambda$, $R = \lambda\tau$, y evalúa $G(Q^*)$ con la función anterior (`["total"]` es la suma). ¿Por qué se separa en una función aparte? Porque el EOQ con descuentos necesita evaluar G en Q que **no** son el óptimo (los bordes de los tramos), y porque las gráficas dibujan cada costo por separado.

Con los números de la empresa

$Q^* = \sqrt{2 \cdot 150{,}000 \cdot 5{,}325.52/97.5} = 4{,}048$ unidades, $T^* = 0.76$ meses, $R = \lambda\tau = 888$, $G =$ \$21,164,214 al mes.

4.2 Con descuentos

La función G da saltos en los quiebres, así que no se puede derivar sobre toda la función. Se resuelve tramo por tramo y al final se comparan los candidatos.

```python
def resolver_eoq_con_desc_devuelve_mejor_y_candidatos(demanda, costo_pedido, tramos,
    tasa_manejo, lead_time):
    candidatos = []

    # Pasos 1 y 2: resolver cada tramo y revisar el caso
    for j in range(len(tramos)):
        precio = tramos[j][1]
        desde, hasta = limites_tramo_devuelve_desde_y_hasta(j, tramos)
        h = h_costo_mantener_una_unidad(tasa_manejo, precio)

        q_tramo = math.sqrt(2 * costo_pedido * demanda / h)   # Q^(j)

        if q_tramo < desde:
            caso = "B"
            q_candidato = desde          # se pide lo mínimo para tener ese precio
        elif q_tramo >= hasta:
```

```
            caso = "C"
            q_candidato = None          # el tramo se descarta
        else:
            caso = "A"
            q_candidato = q_tramo

        if q_candidato is not None:
            costo = costos_eoq_devuelve_ordenar_adquirir_mantener_total(q_candidato,
    demanda, costo_pedido, precio, h)["total"]
        else:
            costo = None

        candidatos.append({"tramo": j, "precio": precio, "q_tramo": q_tramo,
                           "caso": caso, "q_candidato": q_candidato, "costo": costo})

    # Paso 3: escoger el candidato de menor costo (sin contar los descartados)
    mejor_candidato = None
    for candidato in candidatos:
        if candidato["costo"] is None:
            continue                    # pasa al siguiente tramo
        if mejor_candidato is None or candidato["costo"] < mejor_candidato["costo"]:
            mejor_candidato = candidato
    ...
    return mejor, candidatos
```

Léelo así:

1. **Para cada tramo** toma su precio y sus límites, calcula $h_j = Ic_j$ y el EOQ de ese precio, $Q^{(j)} = \sqrt{2K\lambda/h_j}$.
2. **Mira dónde cae** $Q^{(j)}$ respecto a $[Q_{j-1}, Q_j)$:
   - **A, dentro:** sirve tal cual.
   - **B, por debajo:** con ese precio querrías pedir menos, pero el precio solo aplica desde Q_{j-1}. Como G_j es convexa, lo mejor dentro del tramo es el borde Q_{j-1}.
   - **C, por encima:** el óptimo cae en un tramo más barato, que ya se evalúa por su cuenta; este tramo se descarta (`None`).
3. **Guarda** todo en un diccionario y lo agrega a la lista `candidatos`. Se guarda todo, no solo el ganador, porque las tablas y gráficas lo usan.
4. **Escoge el ganador**: recorre los candidatos, salta los descartados (`continue`) y se queda con el de menor costo. La condición `mejor_candidato is None or ...` sirve para que el primer candidato válido entre siempre.

Con los números de la empresa

Tramo	Precio	$Q^{(j)}$	Caso	Candidato	G (mes)
1	\$3,900	4,048	C	—	—
2	\$3,822	4,089	A	4,089	\$20,744,856
3	\$3,705	4,153	B	9,000	\$20,236,629
4	\$3,588	4,221	B	15,000	**\$19,833,977**

Gana el tramo 4: pedir 15,000 unidades. El ahorro en el costo de adquirir pesa más que el inventario extra.

5. Modelo (Q,R) iterativo

5.1 La función de costo

```
def costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total(cantidad, punto_reorden,
    demanda, costo_pedido, costo_unitario, h,
              costo_faltante, media_lead, desviacion_lead):
    """G(Q,R) del (Q,R) separado por componentes. Devuelve un diccionario."""
    faltantes = faltantes_esperados_n_por_ciclo(punto_reorden, media_lead, desviacion_lead
    )   # n(R)

    costo_ordenar = costo_pedido * demanda / cantidad
    costo_adquirir = demanda * costo_unitario
    costo_inventario = h * (punto_reorden - media_lead + cantidad / 2)
    costo_faltantes = costo_faltante * demanda * faltantes / cantidad

    return {"ordenar": costo_ordenar, "adquirir": costo_adquirir,
            "mantener": costo_inventario, "faltantes": costo_faltantes,
            "total": costo_ordenar + costo_adquirir + costo_inventario + costo_faltantes}
```

Es exactamente

$$G(Q,R) = \frac{K\lambda}{Q} + \lambda c + h\Big(R - \mu_\tau + \frac{Q}{2}\Big) + \frac{p\,\lambda\,n(R)}{Q}.$$

- $R-\mu_\tau$ es el inventario de seguridad y $Q/2$ el inventario de ciclo; su suma es el inventario promedio.
- λ/Q son los ciclos por mes y $n(R)$ los faltantes por ciclo; por eso los faltantes cuestan $p\lambda n(R)/Q$.
- Devuelve un diccionario para tener cada costo por separado: las gráficas de barras y de curvas los usan uno por uno.

5.2 El punto de reorden para un Q dado

```
def punto_reorden_dado_q_devuelve_R_z_alfa(cantidad, h, demanda, costo_faltante,
    media_lead, desviacion_lead):
    """Dado Q, calcula R con F(R) = 1 - Q*h/(p*lambda). Devuelve (R, z, F(R))."""
    alfa = 1 - cantidad * h / (costo_faltante * demanda)     # F(R)
    if alfa <= 0:
        # Si Q*h/(p*lambda) >= 1 la fórmula no tiene sentido (el PDF pide Qh/(p*lambda) <
    1)
```

```
        raise ValueError(f"Q*h/(p*lambda) >= 1 con Q = {cantidad:.1f}: no se puede
    calcular R")
    z = norm.ppf(alfa)
    punto_reorden = media_lead + z * desviacion_lead
    return punto_reorden, z, alfa
```

Es la condición $\partial G/\partial R = 0$: $F(R) = 1 - \dfrac{Qh}{p\lambda}$. Con esa probabilidad se busca $z = \Phi^{-1}(F)$ (`norm.ppf`) y se "desestandariza": $R = \mu_\tau + z\,\sigma_\tau$. Si $Qh/(p\lambda) \geq 1$ la probabilidad sería 0 o negativa, así que se lanza un mensaje de error (`raise`): en el (Q,R) sin descuentos el programa se detiene, y en el (Q,R) con descuentos solo se descarta ese tramo.

5.3 La solución completa cuando Q ya se conoce

```
def solucion_qr_dado_q_devuelve_diccionario_completo(cantidad, demanda, costo_pedido,
    costo_unitario, h,
                        costo_faltante, media_lead, desviacion_lead):
    punto_reorden, z, alfa = punto_reorden_dado_q_devuelve_R_z_alfa(cantidad, h, demanda,
    costo_faltante,
                                                    media_lead, desviacion_lead)
    faltantes = desviacion_lead * perdida_normal_L_de_z(z)                    # n(R)
    costos = costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total(cantidad,
    punto_reorden, demanda, costo_pedido, costo_unitario, h,
                        costo_faltante, media_lead, desviacion_lead)
    return {"Q": cantidad, "R": punto_reorden, "ss": punto_reorden - media_lead, "z": z,
            "n(R)": faltantes, "alfa": alfa, "beta": 1 - faltantes / cantidad,
            "precio": costo_unitario, "costos": costos}
```

Con un Q fijo, esta función saca todo lo demás: el R que le corresponde, los faltantes esperados, los costos y los niveles de servicio $\alpha = F(R)$ y $\beta = 1 - n(R)/Q$. Se separó en su propia función porque se usa en **dos** lugares: al final de la iteración (con el Q al que se llegó) y en el caso B de los descuentos (con Q fijo en el borde del tramo).

5.4 La iteración

```
def resolver_qr_sin_desc_devuelve_solucion_con_iteraciones(demanda, desviacion,
    costo_pedido, costo_unitario, tasa_manejo,
                       costo_faltante, lead_time, tolerancia=0.001, max_iteraciones=100):
    h = h_costo_mantener_una_unidad(tasa_manejo, costo_unitario)
    media_lead, desviacion_lead = demanda_horizonte_devuelve_media_y_desviacion(demanda,
    desviacion, lead_time)

    # Paso 1: arrancar con el EOQ
    cantidad = math.sqrt(2 * costo_pedido * demanda / h)
    punto_reorden = None
    iteraciones = []

    # Paso 2: iterar Q -> R -> n(R) -> Q nuevo
    for k in range(max_iteraciones):
        r_nuevo, z, alfa = punto_reorden_dado_q_devuelve_R_z_alfa(cantidad, h, demanda,
    costo_faltante,
                                                    media_lead, desviacion_lead)
```

```
        faltantes = desviacion_lead * perdida_normal_L_de_z(z)                    #
    n(R)
        q_nuevo = math.sqrt(2 * demanda * (costo_pedido + costo_faltante * faltantes) / h)

        iteraciones.append({"k": k, "Q": cantidad, "F(R)": alfa, "z": z,
                            "R": r_nuevo, "n(R)": faltantes})

        # Paso 3: parar cuando Q y R ya casi no cambian
        convergio = (punto_reorden is not None
                     and abs(q_nuevo - cantidad) < tolerancia
                     and abs(r_nuevo - punto_reorden) < tolerancia)
        cantidad = q_nuevo
        punto_reorden = r_nuevo
        if convergio:
            break

    # Con el Q final se recalcula R para que los dos queden consistentes
    solucion = solucion_qr_dado_q_devuelve_diccionario_completo(cantidad, demanda,
    costo_pedido, costo_unitario, h,
                                    costo_faltante, media_lead, desviacion_lead)
    solucion["iteraciones"] = iteraciones      # se agrega la tabla de iteraciones
    return solucion
```

Idea clave

Q depende de R (a través de $n(R)$) y R depende de Q (a través de $F(R)$). No se pueden despejar los dos a la vez, así que se hace un **punto fijo**: con un Q se saca R; con ese R se saca un Q nuevo; y se repite hasta que ninguno cambie.

Paso a paso:

1. Arranca con el EOQ, $Q_0 = \sqrt{2K\lambda/h}$, porque al principio no se conocen los faltantes.
2. En cada vuelta k:
   - con el Q actual saca $F(R)$, z y R (`punto_reorden_dado_q_devuelve_R_z_alfa`);
   - calcula $n(R) = \sigma_\tau L(z)$;
   - calcula el Q nuevo con la otra condición, $Q = \sqrt{2\lambda(K + p\,n(R))/h}$;
   - guarda la fila en `iteraciones`.
3. Para cuando Q y R cambian menos que la tolerancia (0.001 unidades). La condición `punto_reorden is not None` evita comparar en la primera vuelta, cuando todavía no hay un R anterior. `break` sale del `for`.
4. Al final llama a `solucion_qr_dado_q_devuelve_diccionario_completo` con el último Q: recalcula R, los costos y los niveles de servicio que **resultan**, $\alpha = F(R^*)$ y $\beta = 1 - n(R^*)/Q^*$, y agrega la tabla de iteraciones.

Con los números de la empresa

k	Q	$F(R)$	z	R
0	4,048.0	0.9753	1.965	1,928.2
1	4,243.1	0.9741	1.945	1,917.5
2	4,253.5	0.9740	1.944	1,916.9
3	4,254.0	0.9740	1.944	1,916.9

En la vuelta 0: $F(R) = 1 - \dfrac{4{,}048.0 \times 97.5}{3{,}000 \times 5{,}325.52} = 0.9753$, entonces $z = 1.965$ y $R = 887.6 + 1.965 \times 529.5 = 1{,}928$. Converge en pocas vueltas a $Q^* = 4{,}254$, $R^* = 1{,}917$, con inventario de seguridad $ss = 1{,}029$, $\alpha = 0.974$ y $\beta = 0.9988$. Costo: \$21,284,666 al mes.

Ojo

El nivel de servicio **no** se impone: α es el que sale de las iteraciones (el que minimiza costos). La exigencia de 95 % de la empresa se agregará después como restricción.

5.5 Con descuentos

La lógica es la misma del EOQ con descuentos, pero el "óptimo de cada tramo" ahora es un (Q,R) completo:

```
for j in range(len(tramos)):
    precio = tramos[j][1]
    desde, hasta = limites_tramo_devuelve_desde_y_hasta(j, tramos)
    h = h_costo_mantener_una_unidad(tasa_manejo, precio)

    # Paso 1: resolver el (Q,R) del tramo con su precio (sin mirar los límites)
    try:
        solucion_tramo = resolver_qr_sin_desc_devuelve_solucion_con_iteraciones(demanda,
    desviacion, costo_pedido, precio,
                                                tasa_manejo, costo_faltante, lead_time)
        q_tramo = solucion_tramo["Q"]
    except ValueError:                          # Qh/(p*lambda) >= 1 en la iteración: ese
    tramo no tiene solución
        solucion_tramo = None
        q_tramo = math.inf                      # cae por encima: el tramo se descarta (
    caso C)

    # Paso 2: revisar si Q^_j cae dentro del tramo
    if q_tramo < desde:
        caso = "B"
        cantidad = desde                        # Q fijo en el borde
        solucion = solucion_qr_dado_q_devuelve_diccionario_completo(cantidad, ...)   # con
     Q fijo, R sale directo (sin iterar)
    elif q_tramo >= hasta:
        caso = "C"
        solucion = None
    else:
        caso = "A"
        solucion = solucion_tramo
```

- **Reutiliza** `resolver_qr_sin_desc_devuelve_solucion_con_iteraciones` pasándole el precio del tramo. Así no se repite la iteración: dentro de un tramo el precio es fijo y el problema es un (Q,R) normal. Si la iteración de un tramo llega a $Qh/(p\lambda) \geq 1$ (no existe R), ese tramo se descarta (se trata como caso C) y se sigue con los demás.
- En el **caso B**, Q queda fijo en el borde Q_{j-1} y R sale **directo** con $F(R) = 1 - Q_{j-1}h_j/(p\lambda)$, sin iterar: como Q ya no se mueve, no hay nada que iterar. Se usa `solucion_qr_dado_q_devuelve_diccionario_completo`, la misma función con la que termina la iteración.
- Al final, igual que en el EOQ, se escoge el candidato de menor costo total. El costo de adquirir λc_j **sí** cuenta, porque es lo que diferencia a los tramos.

Con los números de la empresa

Tramo	Precio	$\hat{Q}_j$	Caso	Q	R	G (mes)
1	\$3,900	4,254	C	—	—	—
2	\$3,822	4,295	A	4,295	1,919	\$20,863,098
3	\$3,705	4,358	B	9,000	1,748	\$20,336,928
4	\$3,588	4,425	B	15,000	1,617	**\$19,921,138**

Gana el tramo 4. Fíjate que R baja a medida que Q sube: con pedidos más grandes hay menos ciclos al mes, así que cada faltante "pesa" menos y conviene menos inventario de seguridad (α queda en 0.916).

6. Modelo (S,T) iterativo

6.1 Costo para un T fijo

```
def mejor_s_dado_t_devuelve_solucion_o_None(periodo, demanda, desviacion, costo_pedido,
    costo_unitario, tasa_manejo,
                costo_faltante, lead_time):
    h = h_costo_mantener_una_unidad(tasa_manejo, costo_unitario)

    # a. Demanda en el horizonte de protección T + tau
    media_horizonte, desviacion_horizonte = demanda_horizonte_devuelve_media_y_desviacion(
    demanda, desviacion,
                                                                periodo + lead_time)
    # b. F(S) = 1 - h*T/p  ->  z
    alfa = 1 - h * periodo / costo_faltante
    if alfa <= 0:
        return None
    z = norm.ppf(alfa)

    # c. S = mu_{T+tau} + z * sigma_{T+tau}
    nivel_maximo = media_horizonte + z * desviacion_horizonte

    # d. n(S) = sigma_{T+tau} * L(z)
    faltantes = desviacion_horizonte * perdida_normal_L_de_z(z)

    # e. G(S,T)
```

```
    costos = costos_st_devuelve_ordenar_adquirir_mantener_faltantes_total(nivel_maximo,
    periodo, demanda, costo_pedido, costo_unitario, h,
                        costo_faltante, media_horizonte, desviacion_horizonte)
    return {"T": periodo, "S": nivel_maximo, "ss": nivel_maximo - media_horizonte, ...}
```

Para un T dado, S sale **directo** (sin iterar), siguiendo los pasos a–e del algoritmo:

a. El pedido de hoy debe cubrir hasta que llegue el **siguiente** pedido: horizonte $T + \tau$.

b. Condición $\partial G/\partial S = 0$: $F(S) = 1 - hT/p$. Si no se cumple $hT/p < 1$, devuelve `None` para que la búsqueda salte ese T.

c. $S = \mu_{T+\tau} + z\,\sigma_{T+\tau}$.

d. $n(S) = \sigma_{T+\tau} L(z)$.

e. El costo, con `costos_st_devuelve_ordenar_adquirir_mantener_faltantes_total`, que es $G(S,T) = \dfrac{K}{T} + \lambda c + h\Big(S - \mu_{T+\tau} + \dfrac{\lambda T}{2}\Big) + \dfrac{p\,n(S)}{T}$.

`costos_st_devuelve_ordenar_adquirir_mantener_faltantes_total` se escribe igual que `costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total`. La diferencia es que aquí hay un pedido cada T, así que ordenar cuesta K/T y los faltantes $p\,n(S)/T$.

6.2 La búsqueda del mejor T

```
def resolver_st_sin_desc_devuelve_solucion_con_T_probados(demanda, desviacion,
    costo_pedido, costo_unitario, tasa_manejo,
                        costo_faltante, lead_time, paso):
    h = h_costo_mantener_una_unidad(tasa_manejo, costo_unitario)
    datos = (demanda, desviacion, costo_pedido, costo_unitario, tasa_manejo,
             costo_faltante, lead_time)
    probados = {}      # guarda cada T evaluado para no calcularlo dos veces

    def costo_total_con_T_igual_k_pasos(k):
        """Costo total con T = k * paso (infinito si ese T no es válido)."""
        if k not in probados:
            probados[k] = mejor_s_dado_t_devuelve_solucion_o_None(k * paso, *datos)
        if probados[k] is None:
            return math.inf
        return probados[k]["costos"]["total"]

    # Paso 1: punto de partida T0 = EOQ / lambda, redondeado a un múltiplo de 'paso'
    t_inicial = math.sqrt(2 * costo_pedido / (demanda * h))
    k = max(1, round(t_inicial / paso))

    # Paso 2: decidir hacia qué lado baja el costo
    if costo_total_con_T_igual_k_pasos(k + 1) < costo_total_con_T_igual_k_pasos(k):
        direccion = 1
    elif k > 1 and costo_total_con_T_igual_k_pasos(k - 1) <
    costo_total_con_T_igual_k_pasos(k):
        direccion = -1
    else:
        direccion = 0                    # T0 ya es el mínimo
```

```
    # Paso 3: avanzar mientras el costo siga bajando
    while direccion != 0:
        siguiente = k + direccion
        if siguiente < 1 or costo_total_con_T_igual_k_pasos(siguiente) >=
    costo_total_con_T_igual_k_pasos(k):
            break
        k = siguiente
    ...
```

- T no puede ser cualquier número: se revisa cada cierto número de días. Por eso T es un múltiplo de `paso` y el código trabaja con `k`, el número de pasos ($T = k \cdot$`paso`). Para la empresa `paso = 1/30` meses (un día).
- `costo_total_con_T_igual_k_pasos(k)` es una función **dentro** de otra. Evalúa $T = k \cdot$`paso` y guarda el resultado en el diccionario `probados` para no repetir cuentas. Si el T no es válido le pone costo ∞, así nunca se escoge.
- Arranca cerca de $T_0 = \sqrt{2K/(\lambda h)}$, que es el EOQ dividido por λ.
- Mira si el costo baja hacia la derecha (T más grande) o hacia la izquierda, y camina en esa dirección hasta que el costo empiece a subir. Como G es convexa en T, el primer punto donde deja de bajar es el mínimo.

Con los números de la empresa

$T_0 = \sqrt{2 \cdot 150{,}000/(5{,}325.52 \cdot 97.5)} = 0.76$ meses $= 22.8$ días, que se redondea a $k = 23$, es decir $T = 23/30 = 0.767$ meses.

k (días)	T (meses)	S	G (mes)
18	0.6000	6,427	$21,444,417
19	**0.6333**	**6,629**	**$21,443,455**
20	0.6667	6,830	$21,443,700
21	0.7000	7,031	$21,444,970
22	0.7333	7,230	$21,447,117
23	0.7667	7,428	$21,450,019
24	0.8000	7,626	$21,453,575

La búsqueda arranca en 23 días. Hacia 24 el costo sube y hacia 22 baja, así que camina hacia T menores (22, 21, 20, 19). En 18 el costo vuelve a subir, así que se queda en $k = 19$: $T^* = 19$ días $= 0.6333$ meses y $S^* = 6{,}629$. Probó 7 valores de T. Al revisar todos los días de 1 a 400 el mejor es también el día 19.

6.3 Con descuentos

Igual que en (Q,R), con una diferencia: en (S,T) lo pedido cambia en cada revisión, así que el tramo de precio se decide con el **pedido promedio** λT.

```
solucion_tramo = resolver_st_sin_desc_devuelve_solucion_con_T_probados(demanda, desviacion
    , costo_pedido, precio,
                                    tasa_manejo, costo_faltante, lead_time, paso)
t_tramo = solucion_tramo["T"]                  # T^_j
```

```
pedido_promedio = demanda * t_tramo           # lambda * T^_j

if pedido_promedio < desde:
    caso = "B"
    # Borde inferior: el menor T (múltiplo de 'paso') con lambda*T >= Q_{j-1}.
    k_borde = math.ceil(desde / (demanda * paso))
    solucion = mejor_s_dado_t_devuelve_solucion_o_None(k_borde * paso, ...)
elif pedido_promedio >= hasta:
    caso = "C"
    solucion = None
else:
    caso = "A"
    solucion = solucion_tramo
```

En el caso B se busca el T más pequeño cuyo pedido promedio alcance el tramo: $\lambda T \geq Q_{j-1}$, o sea $T \geq Q_{j-1}/\lambda$. Como T va en pasos, se redondea **hacia arriba** con `math.ceil`. Por ejemplo, para el tramo 2: $4{,}000/(5{,}325.52/30) = 22.53$, que sube a $k = 23$, es decir $T = 23$ días $= 0.7667$ meses. Si se redondeara hacia abajo, el pedido no alcanzaría el tramo.

Con los números de la empresa

Tramo	Precio	Caso	T (meses)	S	G (mes)
1	\$3,900	A	0.6333	6,629	\$21,443,455
2	\$3,822	B	0.7667	7,439	\$21,025,844
3	\$3,705	B	1.7000	12,814	\$20,573,832
4	\$3,588	B	2.8333	19,064	**\$20,206,898**

7. El caso de la empresa

7.1 Los datos

```
datos_empresa = {
    "demanda": np.mean(demanda_historica),                  # lambda = 5,325.52 unidades/
    mes (sin redondear)
    "desviacion": np.std(demanda_historica, ddof=1),        # sigma = 1,297.13 (sin
    redondear)
    "costo_pedido": 150_000,     # K por pedido
    "costo_unitario": 3_900,     # c sin descuento
    "tasa_manejo": 0.30 / 12,    # i mensual = 30% anual / 12  ->  h = 97.5 al mes
    "costo_faltante": 3_000,     # p
    "lead_time": 5 / 30,         # tau = 5 días = 0.167 meses
}
```

- λ y σ **no** se escriben a mano: se calculan de los 46 meses de ventas (`demanda_historica`). `ddof=1` da la desviación **muestral** (divide por $n-1$), igual que `DESVEST.M` en Excel.
- Todo se pasa a meses: la tasa anual se divide por 12 y el lead time de días se divide por 30.
- El guion bajo en `150_000` solo separa los miles; Python lo lee como 150000.

- Las claves del diccionario se llaman igual que los parámetros de las funciones. Por eso se puede escribir `resolver_qr_sin_desc_devuelve_solucion_con_iteraciones(**datos_empresa)`.

Antes de usar los datos, el cuaderno revisa el supuesto de demanda normal y estacionaria (Shapiro-Wilk, prueba t de noviembre–diciembre y regresión de tendencia). Conclusión: la demanda es normal ($p = 0.589$), pero tiene temporada alta en noviembre–diciembre y tendencia creciente. Esto queda como limitación.

7.2 Los 6 modelos con los datos reales

```
datos_sin_precio = dict(datos)
del datos_sin_precio["costo_unitario"]      # con descuento el precio lo dan los tramos

sol_qr_sin_desc = resolver_qr_sin_desc_devuelve_solucion_con_iteraciones(**datos)
sol_qr_con_desc, _ = resolver_qr_con_desc_devuelve_mejor_y_candidatos(tramos=
    tramos_empresa, **datos_sin_precio)
sol_st_sin_desc = resolver_st_sin_desc_devuelve_solucion_con_T_probados(**datos, paso
    =1/30)
...
```

Para los modelos con descuento se hace una copia de los datos sin `costo_unitario`, porque esas funciones no reciben un precio único sino los tramos. El `_` recibe los candidatos, que aquí no se necesitan.

Política	Decisión	Precio	G (mes)
EOQ sin descuento	$Q = 4{,}048$	\$3,900	\$21,164,214
EOQ con descuento	$Q = 15{,}000$	\$3,588	\$19,833,977
(Q,R) sin descuento	$Q = 4{,}254$, $R = 1{,}917$	\$3,900	\$21,284,666
(Q,R) con descuento	$Q = 15{,}000$, $R = 1{,}617$	\$3,588	\$19,921,138
(S,T) sin descuento	$T = 0.6333$, $S = 6{,}629$	\$3,900	\$21,443,455
(S,T) con descuento	$T = 2.8333$, $S = 19{,}064$	\$3,588	\$20,206,898

8. Gráficas del inventario en el tiempo (serruchos)

Idea clave

Los serruchos se dibujan con el **valor esperado** de la demanda: se supone que el inventario baja siempre a ritmo λ. No hay nada aleatorio; son las cuentas del modelo dibujadas en el tiempo. Por eso todos los ciclos son iguales.

Una sola función sirve para las tres políticas, porque las tres se pueden describir con dos números: lo que se pide en cada ciclo, q, y el inventario de seguridad, ss.

Política	Pedido q	Inventario de seguridad ss
EOQ	Q	0
(Q,R)	Q	$R - \lambda\tau$
(S,T)	λT	$S - \lambda(T + \tau)$

```python
def serrucho_esperado_devuelve_puntos_neto_posicion_y_pedidos(q, ss, demanda, lead_time,
    horizonte=HORIZONTE_MESES):
    ciclo = q / demanda
    nivel_al_pedir = ss + demanda * lead_time      # inventario cuando se pide (= R en el
    (Q,R))
    t_neto, neto = [], []
    t_posicion, posicion = [], []
    momentos_pedido = []
    inicio = 0
    while inicio < horizonte:
        fin = inicio + ciclo
        pedido = fin - lead_time                   # se pide tau antes de que llegue el
    siguiente pedido

        # Inventario neto: baja de ss + q a ss
        t_neto += [inicio, fin]
        neto += [ss + q, ss]

        # Posición: igual al neto hasta que se pide; ahí sube q (lo que viene en camino)
        t_posicion += [inicio, pedido, pedido, fin]
        posicion += [ss + q, nivel_al_pedir, nivel_al_pedir + q, ss + q]

        momentos_pedido.append(pedido)
        inicio = fin
    return t_neto, neto, t_posicion, posicion, momentos_pedido
```

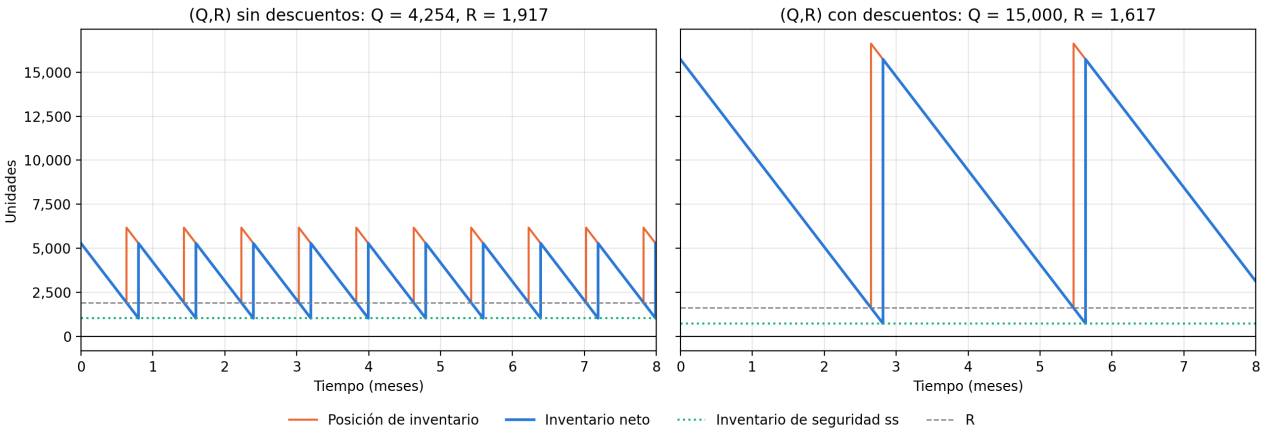
Así se construye un ciclo:

1. El ciclo dura q/λ: lo que se demora en consumirse un pedido.
2. **Inventario neto** (azul): al empezar el ciclo acaba de llegar un pedido, así que está en $ss+q$; baja en línea recta hasta ss al final. Se guardan solo esos dos puntos. Como el ciclo siguiente empieza en el **mismo** tiempo pero en $ss+q$, al unir los puntos matplotlib dibuja la subida vertical: ese es el "diente" del serrucho.
3. **Posición de inventario** (naranja): es el neto más lo que viene en camino. El pedido se hace τ antes de que llegue, en el momento `fin - lead_time`. En ese instante el inventario vale $ss+\lambda\tau$; en el (Q,R) eso es justo R. Ahí la posición sube q de golpe (dos puntos con el mismo tiempo) y sigue bajando hasta $ss+q$ al final del ciclo, donde vuelve a coincidir con el neto.
4. Se repite hasta cubrir 8 meses (`HORIZONTE_MESES`).

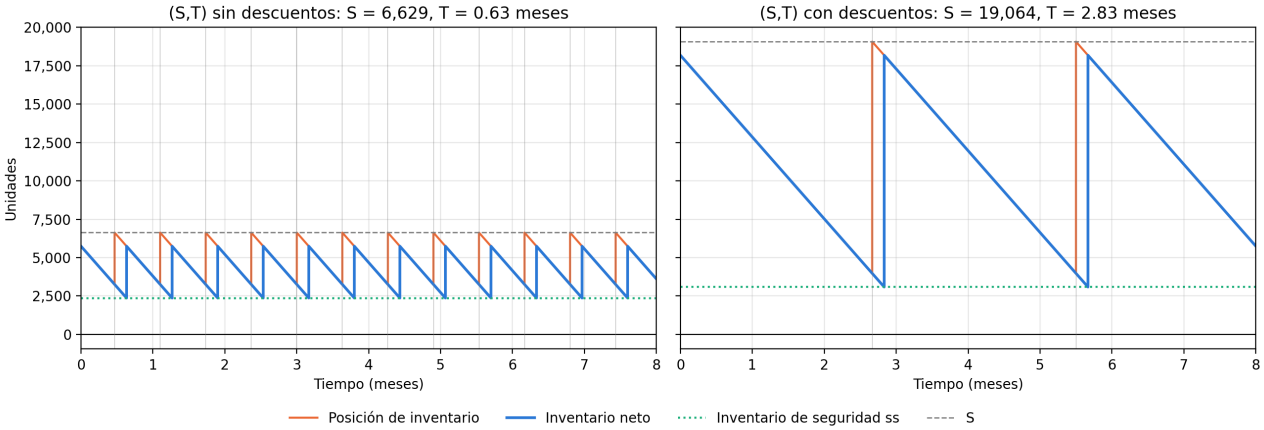
Con los números de la empresa

(Q,R) sin descuento: $q = 4{,}254$, $ss = 1{,}917 - 887.6 = 1{,}029$. Cada ciclo dura $4{,}254/5{,}325.52 = 0.80$ meses. El pedido se hace en $0.80 - 0.17 = 0.63$ meses, cuando el inventario está en $1{,}029 + 887.6 = 1{,}917 = R$. El pedido llega en 0.80 meses, con el inventario en $ss = 1{,}029$, y lo sube a 5,283.

`graficar_serrucho_devuelve_momentos_de_pedido` llama a `serrucho_esperado_devuelve_puntos_neto_posicion_y_pedidos` y dibuja: la posición (naranja), el neto (azul) y una línea punteada verde en ss. Devuelve los momentos de pedido; en el (S,T) son las revisiones, y ahí se dibujan las líneas verticales grises.



Serrucho del (Q,R): la posición toca R, se pide, y τ después sube el inventario neto.



Serrucho del (S,T): en cada revisión (línea gris) la posición sube hasta S y se piden λT unidades.

Ojo

Se supone τ menor que el ciclo (un solo pedido en camino). Con la empresa se cumple de sobra: $\tau = 0.17$ meses y el ciclo más corto es de unos 0.63 meses.

9. Superficies 3D de costo

9.1 La rejilla

```
def costo_en_rejilla_qr_devuelve_matriz_G(Q, R, datos, precio):
    """G(Q,R) (en millones) en cada casilla de las matrices Q y R, con el precio dado."""
    h = h_costo_mantener_una_unidad(datos["tasa_manejo"], precio)
    media_lead, desviacion_lead = demanda_horizonte_devuelve_media_y_desviacion(datos["
    demanda"], datos["desviacion"],
                                                        datos["lead_time"])
    G = np.zeros(Q.shape)                          # matriz vacía para guardar los costos
    for i in range(Q.shape[0]):
        for j in range(Q.shape[1]):
```

```
            costos = costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total(Q[i, j],
     R[i, j], datos["demanda"], datos["costo_pedido"],
                                precio, h, datos["costo_faltante"],
                                media_lead, desviacion_lead)
            G[i, j] = costos["total"] / 1e6   # en millones de pesos
    return G


def superficie_qr_devuelve_matrices_Q_R_G(datos, precio, valores_q, desfases_z):
    h = h_costo_mantener_una_unidad(datos["tasa_manejo"], precio)
    media_lead, desviacion_lead = demanda_horizonte_devuelve_media_y_desviacion(datos["
    demanda"], datos["desviacion"],
                                                                                datos["
    lead_time"])
    Q = np.zeros((len(desfases_z), len(valores_q)))     # matriz de Q (se llena abajo)
    R = np.zeros_like(Q)                                # matriz de R, del mismo tamaño
    for j, q in enumerate(valores_q):                   # para cada Q
        r_centro, _, _ = punto_reorden_dado_q_devuelve_R_z_alfa(q, h, datos["demanda"],
    datos["costo_faltante"],
                                                            media_lead,
    desviacion_lead)   # el mejor R de ese Q
        for i, desfase in enumerate(desfases_z):        # para cada desfase (en
    desviaciones)
            Q[i, j], R[i, j] = q, r_centro + desfase * desviacion_lead    # R = mejor R +
    desfase * sigma_tau
    G = costo_en_rejilla_qr_devuelve_matriz_G(Q, R, datos, precio)   # costo en cada punto
    return Q, R, G
```

Una superficie necesita una altura para cada punto del piso. Aquí el piso es (Q, R) y la altura es $G(Q, R)$.

1. `np.linspace(q_opt / 2, 2 * q_opt, 50)` da 50 valores de Q igualmente espaciados entre la mitad y el doble de Q^* (de 2,127 a 8,508).

2. **Para cada** Q se calcula su mejor R (con `punto_reorden_dado_q_devuelve_R_z_alfa`) y se toman 40 valores de R desde ese mejor R menos 2 desviaciones hasta ese mejor R más 2 desviaciones. La desviación es $\sigma_\tau = \sigma\sqrt{\tau} \approx 529.5$, así que la franja mide $\pm$1,059 unidades alrededor del mejor R. Quedan 2,000 puntos (Q, R), 40 por cada uno de los 50 Q.

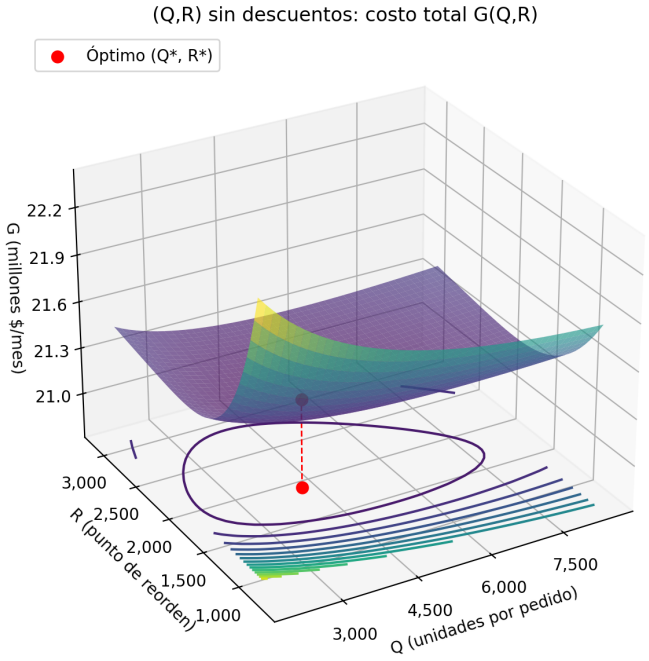
3. `costo_en_rejilla_qr_devuelve_matriz_G` recorre las 2,000 casillas con un doble `for` y en cada una calcula G con la **misma** `costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total` de la sección 5. No hay fórmula nueva: es la función objetivo evaluada muchas veces. Se divide por 10^6 para graficar en millones. Se separó de `superficie_qr_devuelve_matrices_Q_R_G` porque las superficies por tramo (más abajo) también la usan.

9.2 El dibujo

```
ejes = figura.add_subplot(projection="3d")              # ejes en 3D
ejes.plot_surface(Q, R, G, cmap="viridis", alpha=0.65)   # la superficie
piso = G.min() - (G.max() - G.min()) * 0.5   # 50% del rango de G
ejes.contour(Q, R, G, levels=15, cmap="viridis", offset=piso)   # curvas de nivel en el
    piso
ejes.scatter(q_opt, r_opt, g_opt, color="red", s=50)    # el óptimo
```

```
ejes.view_init(elev=28, azim=-120)                    # ángulo de la cámara
```

`plot_surface` une los 2,000 puntos en una superficie coloreada según la altura. `contour` con `offset=piso` proyecta 15 curvas de nivel (G constante) en un plano debajo, como una sombra. Los puntos rojos marcan (Q^*, R^*) arriba y abajo, unidos por una línea punteada. `view_init` solo cambia desde dónde se mira.



9.3 La superficie del (S,T)

En el (S,T) sin descuento, T va de la mitad al doble de T^* (de 0.32 a 1.27 meses). Si se usara un rectángulo fijo de S, con T grande y S pequeño los faltantes se dispararían (unos 31 millones) y taparían el valle. Por eso `superficie_st_devuelve_matrices_T_S_G` dibuja una **franja**: para cada T calcula su mejor S (con `mejor_s_dado_t_devuelve_solucion_o_None`) y toma 40 valores de S desde 2 desviaciones $\sigma_{T+\tau}$ por debajo hasta 2 por encima, y calcula G en cada punto con `costo_en_rejilla_st_devuelve_matriz_G`, que es igual a la del (Q,R) (cambia solo la función de costo que llama). El punto rojo es el óptimo. El ángulo de la cámara (elevación 18, giro 190) se escogió mirando una versión interactiva de la gráfica.

Las superficies **por tramo** (con descuento) usan la misma idea (una franja alrededor del mejor R o S), pero cada tramo se dibuja solo en el rango donde aplica su precio y de un color. Se ve una escalera: cada tramo más abajo que el anterior. La siguiente subsección explica **por qué cada pedazo llega hasta donde llega**.

9.4 ¿Hasta dónde se grafica cada pedazo? (superficies por tramo)

Cada superficie tiene tres ejes, y cada uno se limita por una razón distinta:

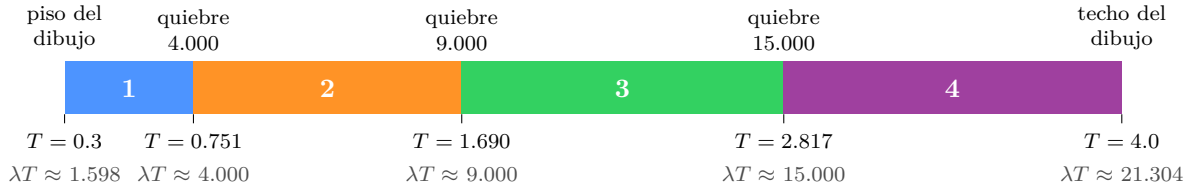
Eje	Qué es	Qué lo limita
Primero	Q en (Q,R); T en (S,T)	**El tramo de precio.** Cada pedazo va de un quiebre al siguiente.
Segundo	R en (Q,R); S en (S,T)	**Una franja de ± 2 desviaciones** alrededor del mejor R (o S) de cada valor del primer eje.
Vertical	G	**Nada:** es el resultado, el costo calculado en cada punto.

Primer eje: lo corta el precio. El precio c_j solo vale si el pedido está en $[Q_{j-1}, Q_j)$. Por eso cada pedazo se dibuja solo ahí.

- En (Q,R) el eje es Q, así que el pedazo del tramo 2 va de 4,000 a 8,999 unidades.
- En (S,T) el eje es T y el pedido promedio es λT. Entonces

$$\lambda T \in [Q_{j-1}, Q_j) \iff T \in \Big[\frac{Q_{j-1}}{\lambda},\ \frac{Q_j}{\lambda}\Big).$$

  Con $\lambda = 5{,}325.52$, el tramo 2 va de $4{,}000/5{,}325.52 = 0.751$ a $9{,}000/5{,}325.52 = 1.690$ meses.
- Por eso cada pedazo empieza justo donde termina el anterior: el **quiebre de precio** es el borde.
- Dos bordes **no** vienen de un quiebre sino del dibujo, porque sin ellos el primer tramo llegaría hasta 0 y el último hasta ∞. En (Q,R) el piso es 1,000 unidades y el techo 20,000 (la capacidad de la bodega). En (S,T) el piso es $T = 0.3$ meses y el techo $T = 4$ meses (unas 21,300 unidades).



Eje T del (S,T), con los números de la empresa. Los números 1 a 4 son los tramos; λT está en miles de unidades.

Segundo eje: una franja, no el tramo. El precio no depende de R ni de S, así que cualquier valor sería válido. Pero para cada Q (o T) solo hay un R (o S) bueno, el que sale de $F(R) = 1 - Qh/(p\lambda)$ (o $F(S) = 1 - hT/p$). Lejos de ese valor, los faltantes (si queda corto) o el inventario (si queda largo) disparan el costo y taparían el valle. Por eso, para cada valor del primer eje se calcula su mejor R (o S) y se dibuja una franja alrededor:

- (Q,R): ± 2 desviaciones, $\pm 2\sigma_\tau$ (unas $\pm 1{,}059$ unidades), alrededor del mejor R.
- (S,T): ± 2 desviaciones, $\pm 2\sigma_{T+\tau}$, alrededor del mejor S.

Como el mejor S crece con T (hay que cubrir más demanda), la franja sube y cada pedazo se ve inclinado. Ejemplo del tramo 2: en $T = 0.751$ el mejor S es 7,346 y $\sigma_{T+\tau} = 1{,}243$, así que la franja va de 4,861 a 9,832. En $T = 1.680$ el mejor S es 12,675 y $\sigma_{T+\tau} = 1{,}763$, y la franja va de 9,150 a 16,201.

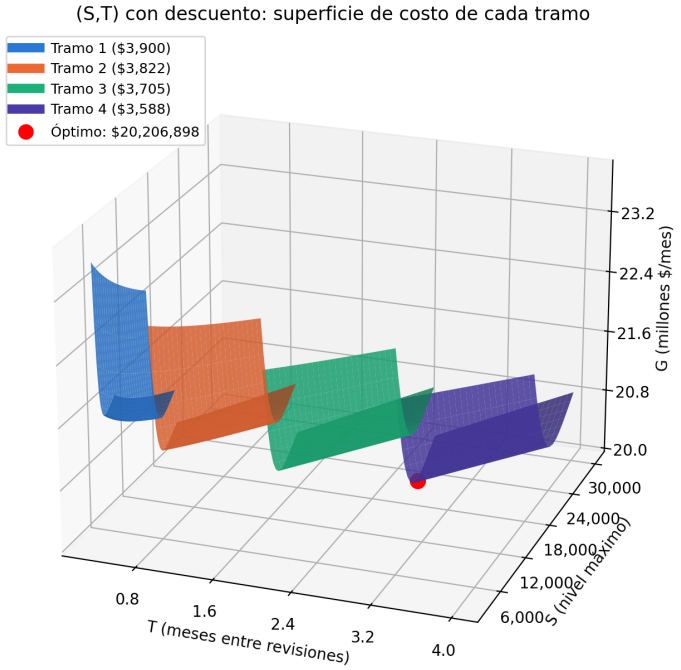
(S,T)	Pedido por revisión λT	T graficado (meses)	S graficado
Tramo 1 ($3,900)	[0, 4,000)	0.30 a 0.74	2,783 a 9,748
Tramo 2 ($3,822)	[4,000, 9,000)	0.75 a 1.68	4,861 a 16,201
Tramo 3 ($3,705)	[9,000, 15,000)	1.69 a 2.81	9,223 a 23,353
Tramo 4 ($3,588)	[15,000, ∞)	2.82 a 3.99	14,492 a 30,542

(Q,R)	Tramo (unidades)	Q graficado	R graficado
Tramo 1 ($3,900)	[0, 4,000)	1,000 a 3,999	872 a 3,274
Tramo 2 ($3,822)	[4,000, 9,000)	4,000 a 8,999	680 a 2,995
Tramo 3 ($3,705)	[9,000, 15,000)	9,000 a 14,999	549 a 2,807
Tramo 4 ($3,588)	[15,000, ∞)	15,000 a 19,999	472 a 2,676

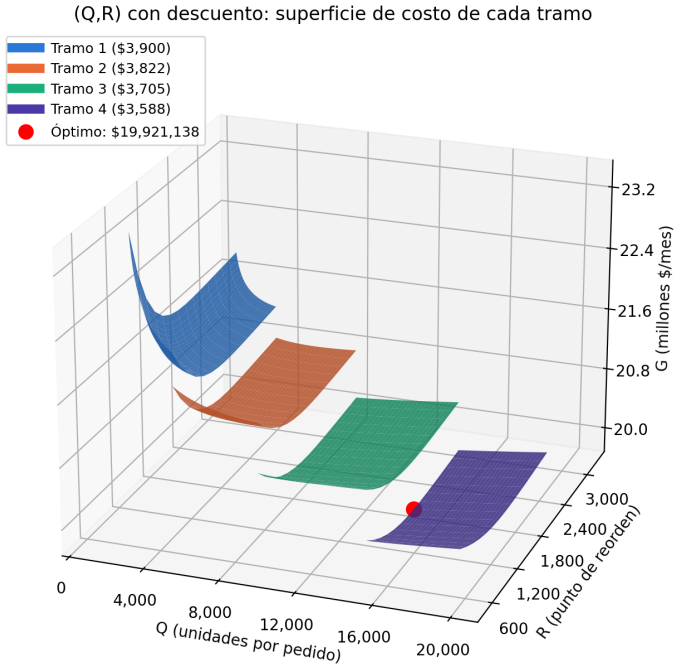
En el código (función `grafica_tramos_3d`) se ve así:

```
desde, hasta = rango_tramo_en_x_devuelve_desde_y_hasta(modelo, j, tramos_empresa,
    datos_empresa["demanda"])
   # límites del tramo en el eje x: en (S,T) son Q_{j-1}/lambda y Q_j/lambda
x = np.linspace(max(desde, 0.3), min(hasta, 4.0) - 0.01, 25)
   # eje T: 25 valores desde el inicio del tramo hasta justo antes del siguiente
X, Y, G = superficie_st_devuelve_matrices_T_S_G(datos_empresa, precio, x, np.linspace(-2,
    2, 15))
   # eje S: franja de ±2 desviaciones alrededor del mejor S de cada T
```

`rango_tramo_en_x_devuelve_desde_y_hasta` convierte los límites del tramo al eje del modelo (en (S,T) divide por λ). `np.linspace` reparte 25 valores dentro de ese rango, con el piso y el techo del dibujo. `superficie_st_devuelve_matrices_T_S_G` (o `superficie_qr_devuelve_matrices_Q_R_G`) devuelve para cada uno el mejor S (o R), los 15 valores de la franja y el costo G de cada punto.



(S,T) con descuento: un pedazo por tramo. Cada pedazo termina donde cambia el precio; la inclinación viene de que el mejor S sube con T.



(Q,R) con descuento: el mismo criterio, con Q en el primer eje y una franja de ± 2 desviaciones de R.

10. Curvas de costo vs Q (o vs T)

Dos funciones de apoyo:

- `costos_en_punto_devuelve_diccionario_o_None(modelo, x, precio, datos)` da los costos de un modelo en un valor del eje x. En EOQ, $x = Q$. En (Q,R), $x = Q$ y se usa el **mejor R para ese Q** (con `punto_reorden_dado_q_devuelve_R_z_alfa`). En (S,T), $x = T$ y se usa el mejor S (con `mejor_s_dado_t_devuelve_solucion_o_None`).
- `curva_costos_devuelve_listas_en_millones` repite lo anterior para todos los valores del eje y guarda cada costo en una lista.

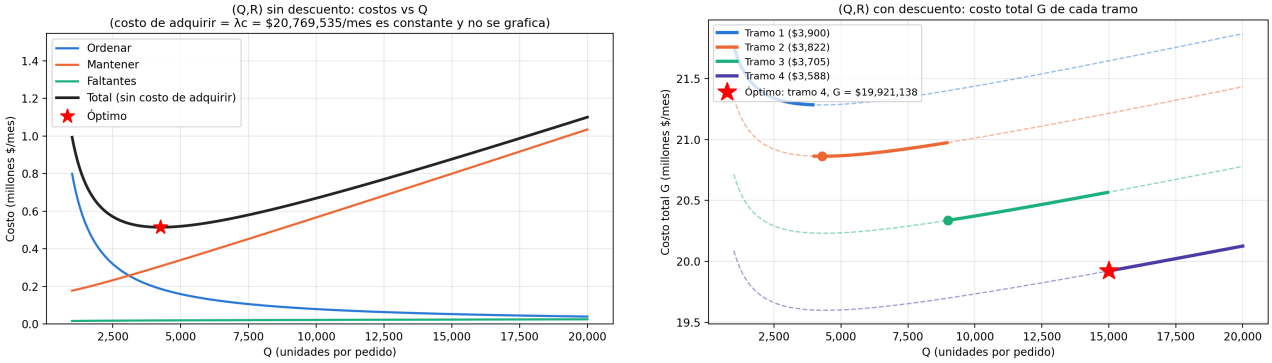
`graficas_de_costo` hace 4 gráficas por modelo:

a) **Sin descuento:** ordenar, mantener, faltantes y su suma. No se grafica el costo de adquirir λc porque es constante y mucho más grande: aplastaría las otras curvas.

b) **Con descuento, por tramo:** lo mismo con el precio de cada tramo; la zona sombreada es donde ese precio aplica y la estrella es el candidato (casos A, B, C).

c) **Costo total de cada tramo:** aquí sí va el costo de adquirir, porque es lo que separa a los tramos. La parte gruesa es donde aplica el precio.

d) **Sin vs con descuento:** la curva con descuento tiene saltos en los quiebres. Por eso se dibuja por pedazos, uniendo solo puntos vecinos del mismo tramo (lo hace `dibujar_curva_con_descuento`, que también usa la gráfica de la política actual).



(Q,R): a la izquierda la gráfica (a); a la derecha la (c). En la (c) se ve que gana el tramo 4.

11. Comparación y política actual

La política actual de la empresa (pedir 1,500 cuando quedan unas 1,488) es un (Q,R) con valores fijos. Por eso no se optimiza nada: se evalúa con las mismas funciones.

```
q_actual = 1500
r_actual = media_lead + 600          # demanda del lead time + 600 de seguridad
politica_actual = {
    "alfa": norm.cdf((r_actual - media_lead) / desviacion_lead),   # F(R)
    "beta": 1 - faltantes_actual / q_actual,
    "costos": costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total(q_actual,
    r_actual, ...),
}
```

La política recomendada es la más barata de las cuatro que consideran faltantes ((Q,R) y (S,T), sin y con descuento). `min` con `key=` escoge, de un diccionario, el nombre cuyo costo total es menor:

```
candidatas = {"(Q,R) sin descuento": sol_qr_sin_desc,
              "(Q,R) con descuento": sol_qr_con_desc,
              "(S,T) sin descuento": sol_st_sin_desc,
              "(S,T) con descuento": sol_st_con_desc}
nombre_recomendada = min(candidatas,
                         key=lambda nombre: candidatas[nombre]["costos"]["total"])
recomendada = candidatas[nombre_recomendada]
recomendada_es_qr = "R" in recomendada     # True si es un (Q,R)
```

Si gana un (Q,R), encaja con la revisión continua que la empresa ya usa; si gana un (S,T), la empresa tendría que pasar a revisión periódica. Las gráficas y el reporte se adaptan solos: `decision_de_politica_devuelve_texto` escribe "Q = …, R = …" o "T = … días, S = …" según el modelo.

Con los números de la empresa

Política actual: $G = \$21{,}796{,}127$ al mes y $\alpha = 0.871$. La más barata de las cuatro es el (Q,R) con descuento: $G = \$19{,}921{,}138$ y $\alpha = 0.916$. El ahorro es de unos \$1.9 millones al mes.

Luego vienen las gráficas de comparación: barras apiladas de costos (`barh` con `left=` para apilar), costo frente a nivel de servicio, la política actual como un punto sobre las curvas del (Q,R) (con un círculo vacío que marca el mismo Q con su mejor R, para mostrar por qué la actual no cae sobre la curva y cuánto cuesta de más; la estrella de la recomendada solo aparece si es un (Q,R)) y los serruchos de actual frente a recomendada (con `graficar_serrucho_devuelve_momentos_de_pedido`).

12. El reporte HTML

La última celda arma `reporte_resultados.html`. No calcula nada nuevo: toma las variables ya calculadas y las pega en texto. Cuatro trucos:

- Los textos de análisis se escriben con `f"..."` y los resultados, así que si cambian los datos, el texto cambia solo.
- `figura_devuelve_bloque_html(archivo, subtitulo, analisis)` busca la imagen en `graficas/`, la convierte a texto con `base64` y la mete dentro del HTML. Así el archivo se puede enviar solo.
- `miles` y `pesos` escriben los números con coma de miles y punto decimal ($21,286,538).
- `poner_coma_de_miles_en_ejes(figura)` hace lo mismo con los números de los ejes de las gráficas (20000 → 20,000). Se llama justo antes de ajustar los márgenes y guardar cada gráfica. En las gráficas 3D deja unos 6 números por eje para que no queden pegados.

13. Mapa: quién llama a quién

Función	Usa a
faltantes_esperados_n_por_ciclo	perdida_normal_L_de_z
resolver_eoq_sin_desc_devuelve_Q_T_R_G	h_costo_mantener_una_unidad, costos_eoq_devuelve_ordenar_adquirir_mantener_total
resolver_eoq_con_desc_devuelve_mejor_y_candidatos	limites_tramo_devuelve_desde_y_hasta, h_costo_mantener_una_unidad, costos_eoq_devuelve_ordenar_adquirir_mantener_total
costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total	faltantes_esperados_n_por_ciclo
solucion_qr_dado_q_devuelve_diccionario_completo	punto_reorden_dado_q_devuelve_R_z_alfa, perdida_normal_L_de_z, costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total
resolver_qr_sin_desc_devuelve_solucion_con_iteraciones	demanda_horizonte_devuelve_media_y_desviacion, punto_reorden_dado_q_devuelve_R_z_alfa, perdida_normal_L_de_z, solucion_qr_dado_q_devuelve_diccionario_completo
resolver_qr_con_desc_devuelve_mejor_y_candidatos	resolver_qr_sin_desc_devuelve_solucion_con_iteraciones (un (Q,R) por tramo) y solucion_qr_dado_q_devuelve_diccionario_completo (caso B)
mejor_s_dado_t_devuelve_solucion_o_None	demanda_horizonte_devuelve_media_y_desviacion, perdida_normal_L_de_z, costos_st_devuelve_ordenar_adquirir_mantener_faltantes_total
resolver_st_sin_desc_devuelve_solucion_con_T_probados	mejor_s_dado_t_devuelve_solucion_o_None (para cada T que prueba)
resolver_st_con_desc_devuelve_mejor_y_candidatos	resolver_st_sin_desc_devuelve_solucion_con_T_probados (un (S,T) por tramo) y mejor_s_dado_t_devuelve_solucion_o_None (caso B)
costo_en_rejilla_qr_devuelve_matriz_G	costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total (en cada casilla de la rejilla)
superficie_qr_devuelve_matrices_Q_R_G	punto_reorden_dado_q_devuelve_R_z_alfa, costo_en_rejilla_qr_devuelve_matriz_G
costo_en_rejilla_st_devuelve_matriz_G	costos_st_devuelve_ordenar_adquirir_mantener_faltantes_total (en cada casilla de la rejilla)
superficie_st_devuelve_matrices_T_S_G	mejor_s_dado_t_devuelve_solucion_o_None, costo_en_rejilla_st_devuelve_matriz_G
graficas_de_costo	costos_en_punto_devuelve_diccionario_o_None, curva_costos_devuelve_listas_en_millones, dibujar_curva_con_descuento
graficar_serrucho_devuelve_momentos_de_pedido	serrucho_esperado_devuelve_puntos_neto_posicion_y_pedidos
poner_coma_de_miles_en_ejes	FormatoConComaDeMiles (el formato de los números de cada eje)

14. Preguntas que te puede hacer tu asesor

¿Dónde está la función objetivo del (Q,R)? En `costos_qr_devuelve_ordenar_adquirir_mantener_faltantes_total`: cada línea es un término de $G(Q, R)$. Es la misma que se usa para la iteración, las curvas, las superficies y la política actual.

¿Por qué el (Q,R) itera y el (S,T) busca? En el (Q,R) las dos condiciones de optimalidad dependen una de la otra (Q de $n(R)$, R de Q), así que se hace punto fijo. En el (S,T), para un T fijo, S sale directo de $F(S) = 1 - hT/p$; lo único que falta es escoger T, y como G es convexa en T basta caminar hasta que el costo suba.

¿Cuándo para la iteración? Cuando Q y R cambian menos de 0.001 unidades entre dos vueltas (`tolerancia`), o a las 100 vueltas como seguro (`max_iteraciones`).

¿Qué pasa en el caso B? El óptimo del tramo cae por debajo del mínimo para obtener ese precio. Como G_j es convexa, lo mejor dentro del tramo es el borde inferior. En (Q,R), con Q fijo, R se calcula una vez; en (S,T) se usa el menor T con $\lambda T \geq Q_{j-1}$.

¿Por qué cada pedazo de las superficies 3D llega hasta ahí? Hay tres ejes y cada uno se limita distinto. El primer eje (Q en (Q,R); T en (S,T)) se corta en los quiebres de precio, porque c_j solo vale si el pedido está en $[Q_{j-1}, Q_j)$; en (S,T) los quiebres se pasan a T dividiendo por λ. El segundo eje (R o S) no lo limita el tramo: es una franja alrededor del mejor valor de cada Q o T ($\pm 2\sigma_\tau$ en (Q,R), $\pm 2\sigma_{T+\tau}$ en (S,T)), para que se vea el valle. G no se limita: es el costo calculado en cada punto. Solo el primer y el último tramo se cortan por el dibujo (piso y techo).

¿Los serruchos son una simulación? No. Usan el valor esperado: el inventario baja a ritmo λ, sube q cuando llega el pedido y toca ss justo antes. Muestran el comportamiento promedio de la política.

¿Se exige el 95 % de servicio? Todavía no. α y β son los que resultan de minimizar costos. La restricción se agregará después.